FLOODGUARD AI: EARLY WARNING AND SMART RESOURCE ALLOCATION FOR URBAN FLOODS

CAPSTONE PROJECT REPORT

Submitted by

S SANJAY KUMAR – 99220040354

P AKILAN – 99220040235

P PANDI VEER VISWA – 99220040342

M MIDHUN– 99220040307

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

COMPUTER SCIENCE AND ENGINEERING

KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

KRISHNANKOIL 626 126

April 2026

DECLARATION

We affirm that the project work titled **"FLOODGUARD AI: EARLY WARNING AND SMART RESOURCE ALLOCATION FOR URBAN FLOODS"** being submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is the original work carried out by us. It has not formed part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

S Sanjay Kumar
99220040354

P Akilan
99220040235

P Pandi Veer Viswa
99220040342

M Midhun
99220040307

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Signature of supervisor
Dr. M. K. Nagarajan
Associate Professor
Department of Computer Science and Engineering

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade

BONAFIDE CERTIFICATE

Certified that this project report **"FLOODGUARD AI: EARLY WARNING AND SMART RESOURCE ALLOCATION FOR URBAN FLOODS"** is the bonafide work of "**S SANJAY KUMAR (99220040354), P AKILAN (99220040235), P PANDI VEER VISWA (99220040342), M MIDHUN (99220040307)"** who carried out the project work under my supervision.

Dr. M. K. Nagarajan
SUPERVISOR
Associate Professor
Computer Science and Engineering
Kalasalingam Academy of Research and Education
Krishnankoil 626126
Virudhunagar District.

Dr. R. Raja Subramanian
HEAD OF THE DEPARTMENT
Associate Professor & Head
Computer Science and Engineering
Kalasalingam Academy of Research and Education
Krishnankoil 626126
Virudhunagar District.

Submitted for the Project Viva-voce examination held on ……

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

We would like to begin by expressing our heartfelt gratitude to the Supreme Power for the immense grace that enabled us to complete this project.

We are deeply grateful to the late **"Kalvivallal" Thiru T. Kalasalingam,** Chairman of the Kalasalingam Group of Institutions, and to late "**Illayavallal" Dr. K. Sridharan**, Chancellor, as well as **Dr. S. Shasi Anand**, Vice President, who has been a guiding light in all our university's endeavours.

Our sincere thanks go to our Vice Chancellor, **Dr. S. Narayanan**, for his inspiring leadership, guidance, and for instilling in us the strength and enthusiasm to work towards our goals.

We would like to express our sincere appreciation to **Dr. P. Deepalakshmi**, Professor & Dean-(SoC), Director Accreditation & Ranking, for her valuable guidance. Our heartfelt gratitude also goes to our esteemed Head of Department, **Dr. R. Raja Subramanian**, whose unwavering support has been crucial to the successful advancement of our project.

We are especially thankful to our Project Supervisor, **Dr. M. K. Nagarajan**, for his patience, motivation, enthusiasm, and vast knowledge, which greatly supported us throughout this work.
Our sincere gratitude also goes to **Dr. T. Manikumar** and **Mr. N. R. Sathis Kumar** Overall Project Coordinators, for their constant encouragement and support in completing this Capstone Project.

Finally, we would like to thank our parents, faculty, non-teaching staff, and friends for their unwavering moral support throughout this journey.

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade

SCHOOL OF COMPUTING

COMPUTER SCIENCE AND ENGINEERING

PROJECT SUMMARY

Project Title	FLOODGUARD AI: EARLY WARNING AND SMART RESOURCE ALLOCATION FOR URBAN FLOODS	
Project Team Members (Name with Register No)	S SANJAY KUMAR – 99220040354 P AKILAN – 99220040235 P PANDI VEER VISWA – 99220040342 M MIDHUN– 99220040307	
Guide Name/Designation	**Dr. M. K. Nagarajan**, Associate Professor, Department of Computer Science and Engineering	
Program Concentration Area	Artificial Intelligence (AI), Internet of Things (IoT), and Disaster Management	
Technical Requirements	Javascript, TailwindCSS, Shadcn/ui,Flask, OpenWeatherMap (for live atmospheric variables), Open-Meteo (for historical data and forecasts), NASA FIRMS (for wildfire alerts), and OpenRouteService (for geocoding and mapping)	
Engineering standards and realistic constraints in these areas		
Area	**Codes & Standards / Realistic Constraints**	**Tick ✓**
Environmental	Since the system is used by "Urban Planners" for disaster management, the hardware sensors providing the distance (water level) data must be rated for extreme weather conditions, ensuring they remain functional during the very flood events they are meant to monitor.	✓
Sustainability	For long-term monitoring by "Municipal Corporations", the remote sensors feeding the database should ideally be solar-powered or utilize low-power wide-area network (LPWAN) technologies to minimize maintenance and battery waste.	✓
Social	Public Accessibility: To serve "Citizens" effectively, the UI must be intuitive and mobile-responsive. The reliance on the Leaflet map and city search requires a level of digital literacy from the local community to benefit from the risk information.	✓

REALISTIC CONSTRAINTS

Environmental:

The primary environmental constraint for this system involves the physical resilience of the hardware required to feed the /distance and /recent_water data routes. Because the project targets Disaster Management Authorities and Urban Planners, any sensors deployed in the field must be ruggedized to withstand the exact environmental stressors they are monitoring, such as heavy torrential rain, high humidity, and potential debris impact during a flood. Furthermore, the system's reliance on geospatial data via Leaflet and OpenWeatherMap creates a dependency on atmospheric conditions; significant storm activity can sometimes interfere with the satellite signals or local network stability required to transmit coordinates (lat, lon) to the /weather_by_coords endpoint. Additionally, the placement of sensors in municipal water systems subjects them to constant chemical exposure and corrosion, necessitating materials that are durable yet environmentally neutral to prevent water contamination.

Sustainability:

Sustainability constraints for this project revolve around Financial Longevity and Technical Maintenance. While the core technologies like Flask, Tailwind CSS, and Shadcn UI are open-source and cost-effective, the operational side of the project relies on external APIs and messaging gateways that may not remain free at scale. The /send_alert route, for instance, incurs per-message costs that must be managed through a sustainable funding model as the user base grows from municipal testers to the general public. Technical sustainability is also hindered by "API Dependency"; if OpenWeatherMap or Geocode.maps.co changes their data structures or pricing tiers, the core functionalities of the "Weather Finder" (such as showSuggestions or displayWeather) would require immediate code refactoring to remain operational. Finally, for the system to be environmentally sustainable, the remote sensors providing live distance values should ideally utilize low-power energy sources, such as solar harvesting, to avoid the logistical and environmental cost of frequent battery replacements.

Engineering Standards:

From a social perspective, the system faces significant constraints regarding Digital Literacy and Public Trust. Since the "Weather Finder" is intended for Local Communities and Citizens, the interface must remain highly intuitive. The current implementation uses a map-

click and search-based trigger system, which assumes users have the technical baseline to interact with interactive web elements and interpret data like wind speed or humidity. A critical social constraint is the potential for Alert Fatigue; if the /send_alert function is triggered by a /flood_risk assessment that lacks high accuracy, repeated false alarms could lead the community to ignore future valid warnings. Language also serves as a barrier, as the current frontend is hardcoded in English, which may limit the effectiveness of the "Weather Finder" in diverse regions where citizens require emergency information in their native tongue to take immediate action.

ABSTRACT

Flooding in urban environments constitutes a critical challenge for modern cities, inflicting severe damage on infrastructure, disrupting socioeconomic stability, and endangering human lives. Conventional flood warning mechanisms, largely dependent on sparse meteorological inputs, frequently suffer from latency and inaccuracy, leading to unreliable or misleading alerts. In this paper, we present Flood Guard AI, an intelligent end-to-end system designed to provide proactive flood risk assessment and early warning. The framework integrates heterogeneous data sources—including real-time meteorological streams, historical hydrological patterns, and satellite-based remote sensing (e.g., NASA FIRMS)—with AI-driven risk modeling to predict and evaluate flood threats. Core components leverage APIs such as OpenWeatherMap and Open-Meteo to ingest live atmospheric variables, while adaptive models compute dynamic flood risk thresholds. To ensure timely dissemination, the system incorporates Server-Sent Events (SSE) for real-time alert broadcasting, thereby guaranteeing immediate notification to authorities and citizens. Experimental evaluations demonstrate that Flood Guard AI not only achieves accurate flood risk estimation but also enables anticipatory resource allocation and coordinated disaster management. By supplying both predictive intelligence and operational responsiveness, Flood Guard AI advances the development of resilient urban infrastructures and data-driven flood mitigation strategies.

Keywords— Flood Risk Prediction, Artificial Intelligence, Urban Floods, Disaster Management, Real-Time Monitoring, Early Warning Systems. Sustainable Development Goals (SDGs)— SDG 9 (Industry, Innovation, and Infrastructure), SDG 11 (Sustainable Cities and Communities), SDG 13 (Climate Action).

TABLE OF CONTENTS

TITLE		PAGE NO.
ABSTRACT		**vii**
LIST OF TABLES		**xii**
LIST OF FIGURES		**xiii**
LIST OF ACADEMIC REFERENCE COURSES		**xiv**
CHAPTER I	**INTRODUCTION**	**1**
1.1	Background and Motivation	**1**
1.2	Problem Statement	**1**
1.3	Objectives of the Project	**2**
1.4	Scope of the Project	**3**
1.5	Methodology Overview	**4**
1.6	Organization of the Report	**5**
CHAPTER II	**LITERATURE REVIEW**	**6**
2.1	Rainfall-Threshold Early Warning Systems	**6**
2.2	Physically-Based Hydrological Models Summary	**6**
2.3	Remote-Sensing Flood Mapping	**7**
2.4	GIS, IoT and Sensor Networks	**7**
2.5	AI and Machine Learning Tecniques	**8**
2.6	Alert Dissemination and Integrated Platforms	**8**
2.7	Hybrid Physical-Data Models	**9**
2.8	IoT and In-Situ Sensor Networks for Local Monitoring	**9**
2.9	Cloud-Based Alert	**9**
2.10	Satellite-based Rainfall Estimate Products	**10**
2.11	Research Gap and Proposed Solution	**10**
CHAPTER III	**SYSTEM ANALYSIS**	**12**
3.1	Requirements Gathering	**12**

3.2	Functional Requirements	**12**
3.3	Non-Functional Requirements	**13**
3.4	Feasibility Study 3.4.1 Technical Feasibility 3.4.2 Operational Feasibility *3.4.3* Economic Feasibility	**14**
3.5	Risk Analysis	**16**
CHAPTER IV	**SYSTEM DESIGN**	**17**
4.1	Overall System Architecture	**17**
4.2	Module Design	**18**
4.2.1	Module 1: Geospatial Weather Engine	**18**
4.2.2	Module 2: Hydrological Monitoring & Alerting	**19**
4.3	Database Design	**19**
4.3.1	Entity-Relationship (ER)	**19**
4.3.2	Database Schema and Table Definitions	**20**
4.4	User Interface Design	**20**
4.4.1	User Flow Diagrams	**21**
CHAPTER V	**IMPLEMENTATION**	**22**
5.1	Technology Stack	**22**
5.1.1	Programming Languages and Tools	**22**
5.2	Implementation of Modules	**22**
5.2.1	Module 1: Weather and Geolocation Services	**22**
5.2.2	Module 2: Hydrological Monitoring and Alerts	**24**
5.3	Integration of Modules	**25**
CHAPTER VI	**TESTING**	**26**
6.1	Testing Methodology	**26**
6.1.1	Unit Testing of Flask Routes and JS Functions	**26**
6.1.2	Integration Testing of API Endpoints	**26**

6.1.3	System and User Acceptance Testing (UAT)	**26**
6.2	Test Cases and Results	**27**
6.3	Bug Tracking and Resolution	**28**
CHAPTER VII	**RESULTS AND DISCUSSION**	**29**
7.1	System Output Screenshots	**29**
7.2	Evaluation Metrics	**30**
7.3	Comparison with Existing Systems	**30**
7.4	Challenges Faced	**31**
7.5	Solutions and Improvements	**31**
CHAPTER VIII	**CONCLUSION & FUTURE SCOPE**	**33**
8.1	Future Improvements	**33**
8.2	Conclusion	**33**
REFERENCES		
PUBLICATION		
CERTIFICATIONS		
PLAGIARISM REPORT		

LIST OF TABLES

TABLES	DETAILS	PAGE NO.
Table 1	Technical Risks	16
Table 2	Database Schema	20

LIST OF FIGURES

FIGURES	DETAILS	PAGE NO.
Figure 1	System Architecture	18
Figure 2	Database Design	20
Figure 3	User Flow Diagram	21
Figure 4	Real-time Weather and Flood Risk Assessment Dashboard	23
Figure 5	Hydrological Trend Analysis and Historical Sensor Records	24

LIST OF ACADEMIC REFERENCE COURSES

S. NO.	COURSE CODE	COURSE NAME
1	211CSE1402	Python Programming
2	211ECE1401	IoT Sensors and Devices
3	212CSE2303	Software Engineering
4	212CSE2305	Database Management System

CHAPTER - I

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

The increasing frequency of extreme weather events and rapid urbanization has made real-time environmental monitoring a critical necessity for public safety. Current disaster management strategies often face challenges due to fragmented data, where weather forecasts and local water level sensors are not integrated into a single, accessible platform. This project is motivated by the need to bridge this gap using a modern full-stack architecture that provides a unified interface for tracking meteorological shifts and local flood risks. By leveraging the Python Flask framework for backend logic and Tailwind CSS with Shadcn UI for a responsive frontend, the system is designed to empower authorities and citizens with actionable data. The primary motivation lies in creating a seamless flow of information—from live "distance" measurements from water sensors to precise weather updates via the OpenWeatherMap API —to significantly reduce the reaction time during natural calamities. This initiative is specifically tailored for Disaster Management Authorities, Municipal Corporations, Urban Planners, and Local Communities. By providing an interactive Leaflet map and an automated SMS alerting mechanism, the project aims to transform raw environmental data into a life-saving communication tool. The integration of high-performance tools like FastAPI or Flask with real-time frontend updates ensures that stakeholders receive the most current information available in memory or the database, fostering a proactive rather than reactive approach to urban resilience.

1.2 PROBLEM STATEMENT AND MOTIVATION

The core challenge addressed by this project is the critical lag between the occurrence of an environmental threat and the dissemination of actionable information to at-risk populations. Traditional disaster response frameworks often suffer from a "data silo" problem, where meteorological data, such as real-time wind speed and temperature, is disconnected from localized hydrological data, such as rising water levels in urban drainage systems. Current systems frequently rely on manual reporting or fragmented web portals that do not provide a unified, geospatial view of the situation.

Specific technical and operational gaps identified include:

- Lack of Real-Time Integration: There is often no direct link between live sensor readings (e.g., distance measurements for water levels) and public-facing weather dashboards, leading to delayed situational awareness.
- Accessibility Barriers: Many existing tools are not designed for the general public, lacking intuitive features like interactive maps or autocomplete city searches that allow "Local Communities and Citizens" to quickly identify risks in their immediate vicinity.
- Inefficient Alerting Mechanisms: Without an automated system to trigger immediate notifications via common communication channels like SMS, authorities must rely on manual broadcasts, which are often too slow during flash flood events.
- Inconsistent Data Availability: Systems often fail to provide a "fail-safe" for data retrieval, missing the ability to toggle between high-speed memory cache for live values and persistent database storage for historical trends.

Consequently, there is an urgent need for a synchronized platform—using a robust Flask backend and a responsive Tailwind CSS frontend—that can process multi-source data and deliver instantaneous alerts to mitigate the impact on human life and urban infrastructure.

1.3 OBJECTIVES OF THE PROJECT

The primary goal of this project is to bridge the gap between complex environmental data and public safety by developing a unified, real-time monitoring and communication platform. By integrating various web technologies and external data sources, the project seeks to provide a reliable tool for disaster mitigation.

The specific technical and operational objectives are as follows:

- Develop a Robust Full-Stack Framework: To build a scalable web application using Python Flask for backend logic and JavaScript for dynamic frontend interactions, ensuring high-speed data processing and a responsive user experience.
- Implement Interactive Geospatial Visualization: To integrate Leaflet.js for an interactive map interface, allowing users to visualize weather patterns and flood risks based on specific geographic coordinates through simple map-click interactions.
- Establish Real-Time Weather Integration: To successfully interface with the OpenWeatherMap API to fetch and display live meteorological data, including

temperature, humidity, wind speed, and weather descriptions, centered initially on the Indian subcontinent.

- Engineer a Hybrid Data Retrieval System: To create a specialized backend route (/distance) that utilizes a "memory-first" approach for live sensor values, falling back to database records only when necessary to ensure zero-latency monitoring of water levels.
- Automate Emergency Communication: To develop a dedicated alerting module (/send_alert) capable of sending instantaneous SMS notifications to citizens and authorities when specific flood risk thresholds are met.
- Enhance User Accessibility through Smart Search: To implement an autocomplete suggestion feature using global geocoding services, enabling users to quickly locate and monitor any city with minimal manual input.
- Provide Historical Data Analysis: To maintain a tracking system (/recent_water) that stores and retrieves the most recent 30 data points, allowing urban planners to analyze trends in rising water levels over time.

1.4 SCOPE OF THE PROJECT

The scope of this project is defined by its functional capabilities, technical boundaries, and the specific user groups it aims to serve within the context of urban disaster resilience.

- Target User Demographic: The system is specifically designed for Disaster Management Authorities, Municipal Corporations, Urban Planners, and Local Communities and Citizens.
- Geographical Focus: While the OpenWeatherMap API and geocoding services provide global coverage, the interactive Leaflet map is initially centered on India ($20.5937, 78.9629$) to address regional flood monitoring needs.
- Technical Framework: The application encompasses a full-stack development scope, utilizing JavaScript for the frontend, Python Flask for the backend, and Tailwind CSS with Shadcn UI for the interface design.
- Data Integration: The project includes the integration of real-time meteorological data—such as temperature, humidity, and wind speed—fetched via external REST APIs.

- Sensor Monitoring: The scope includes the software logic to handle "live" water level distance measurements, utilizing a hybrid retrieval system that checks high-speed memory before falling back to database records.
- Emergency Communication: A core component of the scope is the implementation of an automated SMS alerting system (/send_alert) to notify users of critical risks.
- Historical Tracking: The system is limited to tracking and displaying the 30 most recent hydrological data points for short-term trend analysis.
- Exclusions: The project focuses on the software architecture and data visualization; it does not cover the physical manufacturing of sensors, the long-term archival of multi-year climate data, or the legal enforcement of evacuation orders.

1.5 METHODOLOGY OVERVIEW

The development of the "Weather Finder" system follows a structured, iterative methodology designed to integrate real-time data processing with a user-centric frontend. The approach is divided into four distinct phases: Requirement Analysis, System Architecture & Backend Development, Frontend Engineering, and Integration & Testing.

- Requirement Analysis: This initial phase involved identifying the specific needs of Disaster Management Authorities and Local Communities. Key requirements included the ability to visualize weather data on a map and receive automated alerts based on environmental risks.
- System Architecture & Backend Development: The core of the system was built using the Python Flask framework. During this phase, specialized RESTful endpoints were developed, such as /distance for sensor monitoring and /weather_by_coords for fetching meteorological data. A hybrid data retrieval strategy was implemented to check high-speed memory (_last_live_lock) before querying the database, ensuring minimal latency for live updates.
- Frontend Engineering: The user interface was constructed using JavaScript, Tailwind CSS, and Shadcn UI to ensure responsiveness and modern aesthetics. Leaflet.js was integrated to provide an interactive geospatial layer, allowing the frontend to capture user click coordinates and send them to the Flask backend.
- Integration & Testing: In the final phase, the frontend components were linked to the backend APIs using the Fetch API. This included implementing an autocomplete search function via external geocoding services to improve city lookup speed. Testing

focused on the reliability of the /send_alert route to confirm that SMS notifications are correctly dispatched when risk conditions are simulated.

1.6 ORGANIZATION OF THE REPORT

This report is structured into eight distinct chapters, followed by supporting documentation, to provide a systematic and detailed account of the "Weather Finder" project. The organization is as follows:

- Chapter I: Introduction: Provides the foundational context of the project, including the background and motivation, a clear problem statement, the core objectives, and the scope and methodology used for development.
- Chapter II: Literature Review: Explores related work and similar research in environmental monitoring and disaster management, identifying the specific technological gaps that this project aims to fill.
- Chapter III: System Analysis: Details the requirements gathering process, defining functional and non-functional requirements. It also includes a feasibility study covering technical, operational, and economic constraints, along with a comprehensive risk analysis.
- Chapter IV: System Design: Outlines the overall architecture, module design (frontend, backend, and alerting), and the database structure, including ER diagrams and user interface flow maps.
- Chapter V: Implementation: Focuses on the technology stack, specifically Python Flask, JavaScript, Tailwind CSS, and Shadcn UI, and describes the step-by-step coding and integration of individual modules.
- Chapter VI: Testing: Documents the testing methodology used to ensure system reliability, including unit, integration, and user acceptance testing, along with detailed test cases and bug resolution logs.
- Chapter VII: Results and Discussion: Presents the final system output through screenshots, evaluates the project against specific metrics, compares it with existing systems, and discusses challenges faced and solutions implemented.
- Chapter VIII: Conclusion and Future Scope: Summarizes the project's achievements and outlines potential future enhancements for improved sustainability and broader deployment.

CHAPTER - II

LITERATURE REVIEW

This section reviews related works on flood forecasting, early warning systems, IoT-based monitoring, and cross domain disaster risk assessment. Prior research has explored traditional rainfall threshold methods, physical hydrological models, satellite-based flood detection, AI-driven forecasting, and IoT-enabled early warning platforms. While these approaches provide important insights, they face challenges such as high false alarm rates, computational complexity, lack of generalizability, or limited real-time applicability. The following subsections outline major contributions

I. INTRODUCTION

Flooding is among the most destructive natural disasters across the globe, impacting millions of individuals annually. The rise in intensity of rain, inadequate urban drainage infrastructure, and weather patterns brought about by climate change have worsened urban flooding. In spite of advances and limitations of related works.

2.1 Rainfall-Threshold Early Warning Systems:

Traditional early warning systems for flooding typically issue alerts to river basins and urban catchment areas based on certain rainfall levels or results. These systems are based on predetermined rules and are often applied by national meteorological services and local authorities [1]. Agencies use fixed thresholds of rainfall, for example, to generate a warning when 50 mm/hour or 100 mm has fallen within a 24-hour period. The benefit of traditional systems is that they are easy to implement, simple to read, and have low computational costs. Because of this, they are suitable for regions that do not have many resources; however, these systems are known to have high false positive warning rates because they do not take into account antecedent soil moisture, land use, and other environmental factors [2]. For example, in urban areas with significant coverage of impervious surfaces, even small amounts of rainfall can produce flash flooding, which typical threshold-based systems do not predict well. As such, they may result in people being evacuated unnecessarily, or they may fail to issue warnings for extreme weather events due to the influence of multiple environmental factors.

2.2 Physically-Based Hydrological Models Summary:

Various hydrological or hydraulic models like HEC-HMS, SWAT and MIKE model rainfall-runoff processes and channel flow in order to predict discharge and the extent of

flooding based on a detailed description of the catchment [4]. These models include physical equations that regulate the processes of infiltration, evaporation and routing in channels, thus, providing results which can be physically interpreted; they represent the process dynamics as well as the hydraulics of the floodplain. Advantages include the modeling of complex interactions within the major river basins. Disadvantages include: they need extensive input data and calibration; and they require a great deal of computational power, so there is considerable limitation on their use in real-time, large scale applications [6]. Updating data for land use in these models is time consuming, making them much less useful when sudden monsoon surges occur, especially in regions of India that are currently experiencing rapid urban growth.

2.3 Remote-Sensing Flood Mapping:

Using satellite data from missions like MODIS, Sentinel, and Landsat, Remote Sensing methods combined with optical and SAR data have allowed researchers to determine how much flooding has occurred at an area and what the floods have done over time.Satellite data provides large surface areas to study after flooding has happened, making them a great resource for studying areas of the world where research is not possible, along with being able to provide data about how floods have changed over time, although more research will be needed to determine how long-term flood effects may change areas with regards to their potential for flooding. Additionally, information gathered through satellite-based rainfall products (TRMM, GPM) gives an extra layer of data to supplement bottom-up (ground data) rain gauges in countries where no ground data for rainfall exists. However, while using satellite data is beneficial for studying wide areas, there are still limitations associated with satellite-derived data. These limitations can include: referring to the time between satellite pass-bys and possible blockage of satellite image capture by clouds or other overhead objects, as well as the level of expertise needed to use satellite data has limited some users from accessing and utilizing satellite data to support their research efforts. Remote Sensing has also been used recently to create new studies that utilized both satellite data and on-the-ground research to highlight how loss of vegetation and changes to soil type have created more potential for flooding in areas that are more susceptible to flooding. Although satellite-derived data is extremely helpful for studying large geographical areas of the world in the assessment of floods and flood rehabilitation, they do not provide timely and highly localized data that are necessary when seeking urban flood early warning systems.

2.4 GIS, IoT and Sensor Networks:

GIS-based risk zoning is an approach used to assess vulnerability to flooding from floods. It combines flood maps with population/infrastructure layers, allowing for identification of zones that have high risk of flooding and calculation of evacuation routes and suitability of shelters [5]. The tools that support urban planners with decision-making will provide useful support by enabling spatial visualization of impacts. Additionally, IoT and in-situ sensor networks provide ground truth data at high temporal resolution and deliver real-time observations through river gauges, soil moisture sensors, and rain gauges that are linked using wireless networks [13]. The implementation of these technologies allows for effective local, early warning systems. However, GIS and IoT methodologies have limitations. GIS relies heavily on the use of outdated static base maps, and the deployment and maintenance for IoT systems are costly and usually result in gaps in areas that are remote and lacking resources. There are also difficulties in quantifying change in land use and urbanization for geographically diverse urban communities because of a lack of information about informal settlements [6][5].

2.5 AI and Machine Learning Tecniques:

Using deep learning algorithms for forecasting rainfall/rivers at time of day (time of day) [7][12]. When using ensemble techniques in the machine learning framework, forecasts of floods can provide reliable results from various datasets AI techniques are capable of processing large amounts of data but often have "black-box" characteristics that make them difficult to interpret and diminish trust from end-users who rely on them. Real-time streamflow nowcasting using short-term meteorological predictions with flow-routing and statistical adjustments can help to enhance operational lead times and situational awareness [15], but performance is substantially affected by the accuracy of forecasts and by upstream observations, where the smallest amount of error in one area quickly moves throughout the entire system.

2.6 Alert Dissemination and Integrated Platforms:

We are using cloud distributed alerting systems to communicate urgent information about hazards using SMS (text messaging), dashboards or displays, and pop-up notifications to inform the public or other interested parties (such as emergency responders) [14][15]. These systems allow for fast, widespread communication and quick integration with other services. Multi-agency flood management systems combine multiple agencies' hydrology, geographic information systems (GIS), emergency response and stakeholder coordination systems into a single operational environment [7][11]. Analytical methods, such as crowdsourcing and social media monitoring, can assist in providing rapid situational understanding of a location

using tweets, smartphone applications, and volunteered geographic information [15]. Performance evaluation studies have recommended standardized metrics for evaluating performance of early-warning systems, such as lead time, probability of detection (POD), false alarm ratio (FAR), precision, and area under curve (AUC) [14][16]. However, despite advances in cloud computing and integration of disparate platforms, telecommunication or internet outages during an event make it challenging to access cloud-based and integrative alerts. In addition, crowdsourced data may be noisy, biased, or unreliable without some form of ground truth verification. Barriers to large-scale deployment still arise due to issues of governance and data-sharing as well as privacy-related concerns.

2.7 Hybrid Physical-Data Models:

The integration of hydrological simulations and Machine Learning-based models for residuals/error correction provides specific benefits and challenges for practitioners using these models. The use of both physical and data-driven parameterizations leads to more accurate and generalizable models than either a purely data-driven or purely physical model. This hybrid model type, however, tends to be more complex to develop, requiring careful integration of model components and additional computational resources [17]. Hybrid models also demonstrate improved performance over purely data-driven or physical models when used in mixed urban-rural watershed applications; nevertheless, they are often challenging to calibrate [10].

2.8 IoT and In-Situ Sensor Networks for Local Monitoring:

Use of wireless sensor networks to deploy river level gauges, soil moisture sensors, and rain gauges to provide low-latency, ground-truth observations. The primary benefits of this type of deployment are the high temporal resolution and the ability to measure data directly at the ground, which supports the use of these sensors as early warning systems [11]. Conversely, the costs associated with deployment and maintenance of these wireless sensor networks and their lack of coverage in remote or resource-limited areas make them challenging for widespread use. In India, the deployment of IoT applications in cities such as Chennai has improved local monitoring; however, these deployments will be more challenging to scale to rural areas that are prone to flooding [13].

2.9 Cloud-Based Alert:

Distribution and Mobile Platforms Infrastructure to distribute alerts, visual dashboards, and send Push notifications to public and stakeholders using cloud technology and mobile/text messaging is one of the most effective means to communicate across a wide area. Advantages include being able to quickly and easily communicate across large areas as well

as being able to be integrated into almost any other type of system [15]. Disadvantages include being dependent on telecommunications and/or Internet infrastructure, which can have diminished reliability in disaster scenarios; however, privacy and/or opt-in limits are also limitations. Many users have reported that although cloud technology allows for quick notification after a disaster, if the cellular network is non-functional as a result of extreme weather systems, the user may still not receive an alert [14].

2.10 Satellite-based Rainfall Estimate Products:

Satellite-Derived Rainfall Products for Forecasting as a Forecasting Tool Satellite precipitation estimates (i.e. TRMM, GPM) provide valuable information about freshwater supplies by complimenting existing surface-based monitoring systems, especially in the case of hydrologically ungauged basins and limited data availability (e.g. Brown et al., 2022) [16]. Advantages of using satellite rainfall estimates include: (1) increasing areal coverage of precipitation relative to the area covered by available surface-based measurements, and (2) using these estimates for regional forecast purposes. Disadvantages of satellite rainfall estimates are that they have generally lower precision than local gauge-based surface measurements and that the use of satellite-based estimates requires bias and latency correction after the fact in order to achieve operational reliability (e.g. Akhatar et al., 2022). Satellite-derived rainfall estimate products are certainly useful for assisting how to manage freshwater resources in areas with little to no surface-based measurements; however, obtaining these products requires post-analysis to allow for reliable use operationally [11].

2.11 Research Gap and Proposed Solution:

Current research usually focuses on only single aspects of floodproofing (e.g., rainfall threshold, hydrology modelling, GIS mapping, or separate artificial-intelligence methods) like those found in [1] [2] [4] [17]. Very little work has integrated the heterogeneous live data used for real-time alerts or given a complete means to allocate resources proactively with the use of automated alerts [10] [11] [13]. While real-time alert streaming, continuous integration of live sensor data from sensors and weather APIs, and wildfire data with a scalable architecture for future IoT and ML integration have much room for development [14] [16] [17] FloodGuard AI provides these capabilities directly by combining,live ultrasonic sensor reading,open weather meterological data streams,dynamic risk score assignment, dynamic visualisation of geospatial data, … short-term forecasting … and configurable automated SMS notifications – in real-time on a single production-ready system. The system provides

means for normalising flood risk assessment scores, streaming real-time alerts and providing a scalable architecture to support future growth such as NASA's Fire Information for Resource Management System (FIRMS) or advanced machine Learning models. In addition to achieving zero latency for hybrid data retrieval, relativity aware / context-based assessment of risk and real-world decision support tools, at present, FloodGuard AI successfully brings together all components currently fragmented in prior studies, therefore representing an effective deployable method of building urban flood resilience in India and other local regions.

CHAPTER - III
SYSTEM ANALYSIS

3.1 REQUIREMENTS GATHERING

The requirements gathering process for this disaster management and weather monitoring system was conducted through a multi-dimensional approach to ensure all stakeholders—from government officials to local citizens—were considered. The primary goal was to identify the technical and operational gaps in existing localized flood risk monitoring.

The following methods and sources were utilized to define the system's core needs:

- Stakeholder Identification: Three distinct user groups were identified: Disaster Management Authorities for high-level oversight, Municipal Corporations for urban planning, and Local Communities for individual safety.
- Source Code and Technical Review: Analysis of existing backend logic revealed the need for a hybrid data retrieval system that could handle both high-speed "live" memory values and persistent database records for water level "distance" monitoring.
- External API Capability Mapping: A review of the OpenWeatherMap API was conducted to determine the specific meteorological data points available, such as temperature, humidity, and wind speed, which are essential for the system's weather finder functionality.
- Geospatial Interaction Design: Research into the Leaflet.js library helped define the requirement for an interactive mapping interface where users could click specific coordinates to trigger localized weather queries.
- Emergency Protocol Review: Analysis of disaster response workflows identified the critical need for a POST-based alerting system (/send_alert) that can deliver immediate SMS notifications containing specific risk messages.
- UX/UI Prototyping: Preliminary CSS and HTML structures were analyzed to establish the requirement for a responsive, mobile-friendly dashboard that uses Tailwind CSS and Shadcn UI for clear data visualization.

3.2 FUNCTIONAL REQUIREMENTS

The functional requirements define the specific behaviors and services the Weather Finder and flood monitoring system must provide to satisfy the needs of its target users. These requirements ensure that the interaction between the Python Flask backend and the JavaScript frontend results in actionable data for disaster mitigation.

The core functional requirements are as follows:

- Interactive Map Navigation: The system must provide a Leaflet.js map interface that allows users to click on any geographic coordinate to trigger a weather data request.
- Coordinate-Based Weather Retrieval: Upon receiving a click event, the backend must process a POST request to the /weather_by_coords endpoint and return live data from the OpenWeatherMap API, including temperature, humidity, and wind speed.
- City-Based Search Functionality: Users must be able to input a city name into a search bar to retrieve weather data specifically for that location.
- Automated Location Suggestions: The frontend must implement an autocomplete feature that triggers after two characters are typed, fetching matching city names from an external geocoding service.
- Real-Time Hydrological Monitoring: The system must provide a dedicated route (/distance) that returns the current water level distance, prioritizing live values stored in memory (_last_live_lock) over database records to ensure zero-latency updates.
- Historical Data Visualization: The platform must allow users to view the 30 most recent water level entries via the /recent_water route to analyze short-term environmental trends.
- Emergency SMS Alerting: A robust /send_alert endpoint must be available to dispatch critical messages to specific phone numbers, returning a confirmation status once the SMS is successfully sent.
- Dynamic Data Display: The interface must dynamically update the "weather-box" to show specific icons, temperature in Celsius, and wind speed in m/s without requiring a full page refresh.

3.3. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the quality attributes, design constraints, and performance goals of the Weather Finder and flood monitoring system. These ensure that the platform is not only operational but also reliable, secure, and user-friendly under real-world conditions.

The core non-functional requirements are as follows:

- Performance and Latency: The system must prioritize high-speed data retrieval for critical monitoring, using a "memory-first" approach for the /distance route to ensure that live sensor values are served with minimal delay. The weather interface must update dynamically without requiring a full page reload, maintaining a smooth user experience even during frequent map interactions.
- Scalability and Availability: The backend, powered by Python Flask, must be capable of handling concurrent requests to the /weather_by_coords and /weather_by_city endpoints. The system should remain available even if specific external services encounter issues, returning appropriate error messages (e.g., "weather_fetch_failed") with a 503 status code rather than crashing.
- Responsiveness and Cross-Platform Compatibility: Utilizing Tailwind CSS, the user interface must be fully responsive, ensuring that "Local Communities and Citizens" can access flood risks and weather updates on both mobile devices and desktop computers.
- Reliability and Robustness: The application must implement strict error handling for user inputs and API responses. For instance, if a city is not found or a coordinate is invalid, the system must inform the user through the "weather-box" display rather than failing silently.
- Usability and Accessibility: The interface must be intuitive, featuring a Leaflet map centered on India for quick regional reference and a search bar with autocomplete suggestions that trigger after only two characters to assist users with varying levels of digital literacy.
- Security and Data Integrity: Access to the OpenWeatherMap API and other external services must be secured via server-side keys (like OPENWEATHER_API_KEY) to prevent unauthorized usage or exposure of credentials in the client-side code.
- Maintainability: The codebase should follow a modular structure, separating the JavaScript frontend logic, Tailwind styling, and Flask backend routes to allow for easy updates and the addition of future features like enhanced flood risk algorithms.

3.4. FEASIBILITY STUDY

The feasibility study evaluates the "Weather Finder" and flood monitoring project from technical, operational, and economic perspectives to determine its viability as a sustainable solution for disaster management.

3.4.1 Technical Feasibility

The project is technically viable as it utilizes established, industry-standard technologies and reliable third-party services. The core infrastructure is built on the Python Flask framework, which is highly efficient for managing the RESTful API endpoints required for coordinate-based weather retrieval and sensor data management.

- Data Integration: The system successfully interfaces with the OpenWeatherMap API to fetch real-time meteorological data using authenticated requests.
- Mapping and UI: The use of Leaflet.js provides a mature, open-source solution for interactive geospatial visualization, while Tailwind CSS and Shadcn UI ensure a responsive frontend capable of dynamic content updates.
- Performance Optimization: The implementation of a "memory-first" retrieval system for water level data (_last_live_lock) proves that the system can handle high-frequency sensor updates with minimal latency.
- Communication: The inclusion of a dedicated /send_alert route demonstrates the technical capability to bridge web-based risk assessment with mobile SMS communication.

3.4.2 Operational Feasibility

The system is designed to integrate seamlessly into the existing workflows of Disaster Management Authorities and Municipal Corporations.

- User-Centric Design: Features like city-based search with autocomplete suggestions and map-click triggers make the platform accessible to "Local Communities and Citizens" regardless of their technical expertise.
- Decision Support: By providing both live weather data and the 30 most recent hydrological entries, the system offers urban planners a clear data-driven basis for emergency response and flood risk mitigation.
- Automation: The automated SMS alerting mechanism reduces the need for manual monitoring, allowing authorities to focus on coordination while the system handles the immediate dissemination of risk information.

3.4.3 Economic Feasibility

The economic feasibility of the project is supported by the strategic use of open-source tools and scalable third-party services.

- Development Costs: By utilizing open-source frameworks like Flask, JavaScript, and Tailwind CSS, the initial development costs are significantly minimized compared to proprietary disaster management software.
- Operational Scalability: While external APIs like OpenWeatherMap and SMS gateways may incur costs at high volumes, the current architecture allows for modular scaling.
- Long-term Value: The system provides a cost-effective alternative for municipal-level monitoring, potentially saving significant funds by providing early warnings that reduce property damage and infrastructure loss during flood events

3.5 RISK ASSESSMENT

The Risk Analysis identifies potential obstacles that could impact the deployment, reliability, and accuracy of the Weather Finder and flood monitoring system. Managing these risks is critical for a platform intended for Disaster Management Authorities and Local Communities

Table 1: Technical Risks

Risk Category	Potential Threat	Impact	Mitigation Strategy
Technical	API Downtime/Rate Limiting: Dependency on external services like OpenWeatherMap or Geocode.maps.co.	High: System cannot fetch live weather or city suggestions.	Implement robust error handling with 503 status codes and cache recent weather data to provide "last known" info during outages.
Data Integrity	Sensor Failure: Loss of live "distance" data from hardware monitors.	Critical: Authorities lose real-time visibility into rising water levels.	Use a hybrid retrieval system; if live memory fails, fall back to the latest database record to maintain continuity.
Operational	Alert Latency: Delays in the /send_alert route during high network traffic.	High: Citizens may not receive evacuation notices in time.	Use asynchronous task queues for SMS dispatch and provide instant "status: sent" feedback to the administrator.

*Note: The **Software Engineering (212CSE2303)** concepts of requirement elicitation, feasibility analysis, and risk management have informed the design of our Flask-JavaScript application for disaster mitigation. This ensures that functional requirements like map-based weather retrieval and non-functional requirements like system reliability are systematically validated, ensuring the integration of real-time environmental monitoring into urban planning will be successful.*

CHAPTER - IV

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The "Weather Finder" and flood monitoring system follows a Client-Server Architecture designed for high-speed data processing and real-time user interaction. The architecture is structured to bridge external meteorological services with internal sensor data management.

The system architecture consists of the following key layers:

- Presentation Layer (Frontend): Built using JavaScript, Tailwind CSS, and Shadcn UI, this layer handles user interactions such as map clicks, city searches, and the display of weather metrics. It utilizes the Leaflet.js library to provide an interactive geospatial interface.
- Application Layer (Backend): A Python Flask server acts as the central intelligence of the system. It manages various RESTful routes, including /weather_by_coords for coordinate-based fetching and /distance for hydrological monitoring.
- Data Acquisition Layer: This layer integrates two distinct data streams:
    - External Data: Real-time meteorological information is retrieved from the OpenWeatherMap API. Geocoding and autocomplete suggestions are handled via Geocode.maps.co.
    - Internal Data: Local water level data is managed through a hybrid system that prioritizes high-speed Memory Cache (_last_live_lock) for immediate readings, with a Database (DB) fallback for historical consistency.
- Communication Layer: A dedicated alerting module (/send_alert) interfaces with SMS gateways to push critical flood risk notifications directly to the users' mobile devices.

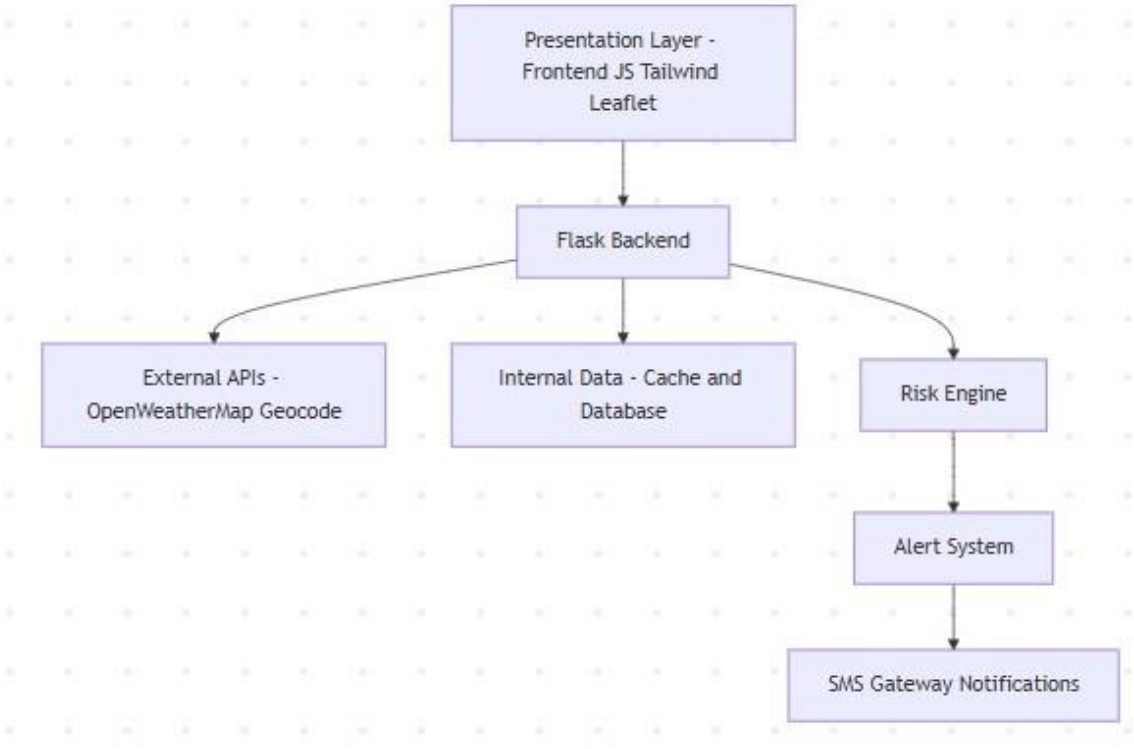


Figure 1: System Architecture

4.2 MODULE DESIGN:

The system is partitioned into specialized modules to ensure high cohesion and maintainability, allowing for independent scaling of weather services, sensor data, and emergency alerts.

4.2.1 Module 1: Geospatial Weather Engine

This module serves as the primary interface for users to interact with environmental data through geographic visualization.

- Map Rendering Interface: Utilizes Leaflet.js to initialize a global map, centered on India by default, providing users with a visual context for data selection.
- Coordinate Processing: Captures latitude and longitude via "click" events on the map surface and transmits them to the Flask backend through a POST request to /weather_by_coords.
- Weather Data Integration: Interfaces with the OpenWeatherMap API to retrieve localized temperature, humidity, and wind speed based on the provided coordinates.
- Dynamic Display Unit: Updates the "weather-box" component in real-time with formatted strings and graphical weather icons without requiring a page refresh.

4.2.2 Module 2: Hydrological Monitoring & Alerting

This module manages the critical sensor data flow and the dissemination of risk-based communications to stakeholders.

- Hybrid Data Retrieval: Implements a /distance route that checks a high-speed memory lock (_last_live_lock) for immediate sensor values before falling back to the database for historical records.
- Historical Trend Tracking: Provides a /recent_water endpoint that fetches the last 30 entries of water level data to allow for short-term pattern analysis.
- Risk Assessment Engine: Evaluates incoming data via a /flood_risk endpoint to determine if environmental thresholds have been exceeded.
- Emergency Messaging Service: Executes a /send_alert function that interfaces with SMS gateways to push critical warnings to "to" and "message" parameters defined in the request body.

4.3 DATABASE DESIGN:

The database for the Weather Finder and flood monitoring system is designed to handle high-frequency sensor data while maintaining a reliable historical log for urban planning and disaster analysis.

4.3.1 ER Diagram

The Entity-Relationship (ER) diagram illustrates the relationship between the core data entities: Sensors, Weather Logs, and Alerts.

- Sensor: Represents the physical hardware devices. Each sensor is linked to multiple distance readings.
- Distance_Reading: Stores the "live" and historical water level values retrieved via the /distance and /recent_water routes.
- Weather_Log: Stores localized weather data (temperature, humidity, wind speed) fetched from the OpenWeatherMap API.
- Alert_Log: Tracks every emergency message sent via the /send_alert route, including the recipient and timestamp.

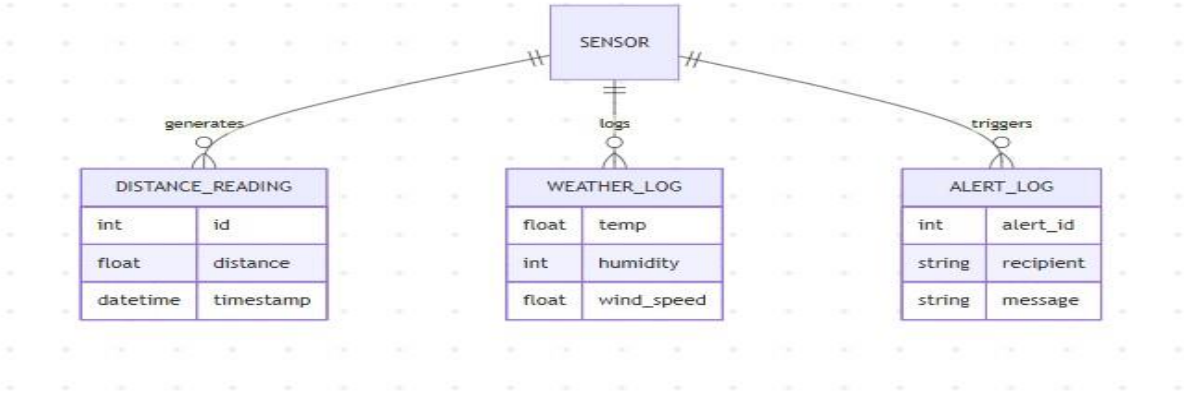


Figure 2: Database Design

4.3.2 Database Schema

The schema is optimized for quick retrieval, specifically supporting the fetch_latest_row() and fetch_recent(30) functions used in the backend.

Table Name	Column Name	Data Type	Description
Water_Levels	id	INT (PK)	Unique identifier for each reading.
	distance	FLOAT	The measured water level value.
	timestamp	DATETIME	The exact time the data was recorded.
	source	VARCHAR	Indicates if data came from "memory" or "db".
Weather_Cache	city_id	INT (PK)	Unique ID for the city or coordinates.
	temp	FLOAT	Temperature in Celsius.
	humidity	INT	Humidity percentage.
	wind_speed	FLOAT	Wind speed in m/s.
Alerts	alert_id	INT (PK)	Unique ID for the sent alert.
	recipient	VARCHAR	The phone number the alert was sent to.
	message	TEXT	The content of the emergency alert.
	status	VARCHAR	Status of the message (e.g., "sent").

4.4 User Interface Design

The user interface (UI) for the Weather Finder is designed to be a single-page dashboard that prioritizes ease of use and immediate data visualization. The layout uses Tailwind CSS for a clean, modern aesthetic and Leaflet.js for interactive mapping.

4.4.1 User Flow Diagrams

The system follows two primary logic flows to ensure users can access information quickly, whether they are searching for a specific location or exploring a map.

- Map Interaction Flow:
    1. The user views the Leaflet map, which is initialized to center on India.
    2. The user clicks a specific point on the map.
    3. A POST request is sent to the /weather_by_coords endpoint with the captured latitude and longitude.
    4. The backend fetches data from the OpenWeatherMap API.
    5. The frontend dynamically updates the "weather-box" with the current temperature, humidity, and wind speed.
- Search and Autocomplete Flow:
    1. The user begins typing a city name into the input field.
    2. Once the input length reaches 2 characters, the showSuggestions function triggers a fetch to the geocoding API.
    3. A dropdown list of matching locations appears.
    4. The user selects a suggestion, which populates the search bar and allows them to click "Search".
    5. The system calls the /weather_by_city endpoint to display the final weather results.
- Emergency Alert Flow (Admin/Authority):
    1. The user (Authority) identifies a risk and accesses the /send_alert interface.
    2. The user inputs the recipient's phone number and the emergency message.
    3. Upon submission, the system executes the send_sms function.
    4. The system returns a "sent" status and a confirmation info block to the UI.

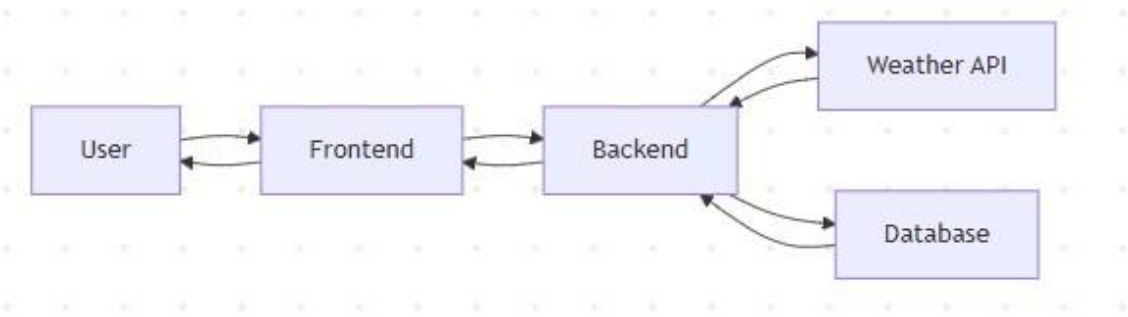


Figure 3: User Flow Diagram

*Note: The **Database Management System (212CSE2305)** concepts of schema normalization, ER modeling, and efficient query execution have informed the design of our backend architecture for the storage of hydrological data and weather logs. By mapping sensor "distance" readings to structured tables and optimizing the retrieval of the 30 most recent records, we ensure the data persistence layer is robust and the transition from memory-cache to disk storage will be successful.*

CHAPTER - V

IMPLEMENTATION

5.1 TECHNOLOGY STACK

The implementation of the Weather Finder and flood monitoring system relies on a coordinated stack of modern web technologies, chosen for their efficiency in handling real-time data and providing a responsive user experience. The architecture separates concerns between a logic-heavy backend and a dynamic, interactive frontend.

5.1.1 Programming Languages and Tools

The core of the application is built using Python for the backend logic and JavaScript for the frontend interactivity. The backend utilizes the Flask web framework, which was selected for its lightweight nature and its ability to easily create RESTful API endpoints for weather data and sensor monitoring. On the frontend, Tailwind CSS and Shadcn UI are used to build a modern, utility-first user interface that ensures the dashboard is visually clear and responsive across different devices.

For geospatial features, the system integrates the Leaflet.js library, an open-source JavaScript tool that enables the interactive map functionality. Data for the system is sourced from multiple external providers: OpenWeatherMap supplies live meteorological data through its API, while Geocode.maps.co is used to power the city autocomplete and search suggestion features. The hardware integration for water level monitoring is managed through a hybrid approach in the Flask backend, utilizing memory-based variables for live "distance" values and a database for persistent historical storage

5.2 IMPLEMENTATION OF MODULES

The implementation of the system is divided into functional modules that handle specific tasks, from weather data processing to emergency communication.

5.2.1 Module 1: Weather and Geolocation Services

This module handles the integration between the user interface and external meteorological data. It is implemented using **JavaScript** on the frontend to capture user input and Python Flask on the backend to process API requests.

- Coordinate-Based Weather: The system uses a map.on("click") listener in Leaflet.js to capture the latitude and longitude of a user's click. These coordinates are sent via a POST request to the /weather_by_coords endpoint, which uses the OpenWeatherMap API to return localized data.
- City Search and Autocomplete: A showSuggestions function is implemented to trigger when a user types at least two characters. It fetches data from https://geocode.maps.co/search and dynamically populates a suggestions box, allowing users to select and search for specific cities.
- Data Visualization: The displayWeather function processes the JSON response from the backend, updating the DOM to show the city name, weather icons, temperature in Celsius, humidity, and wind speed.

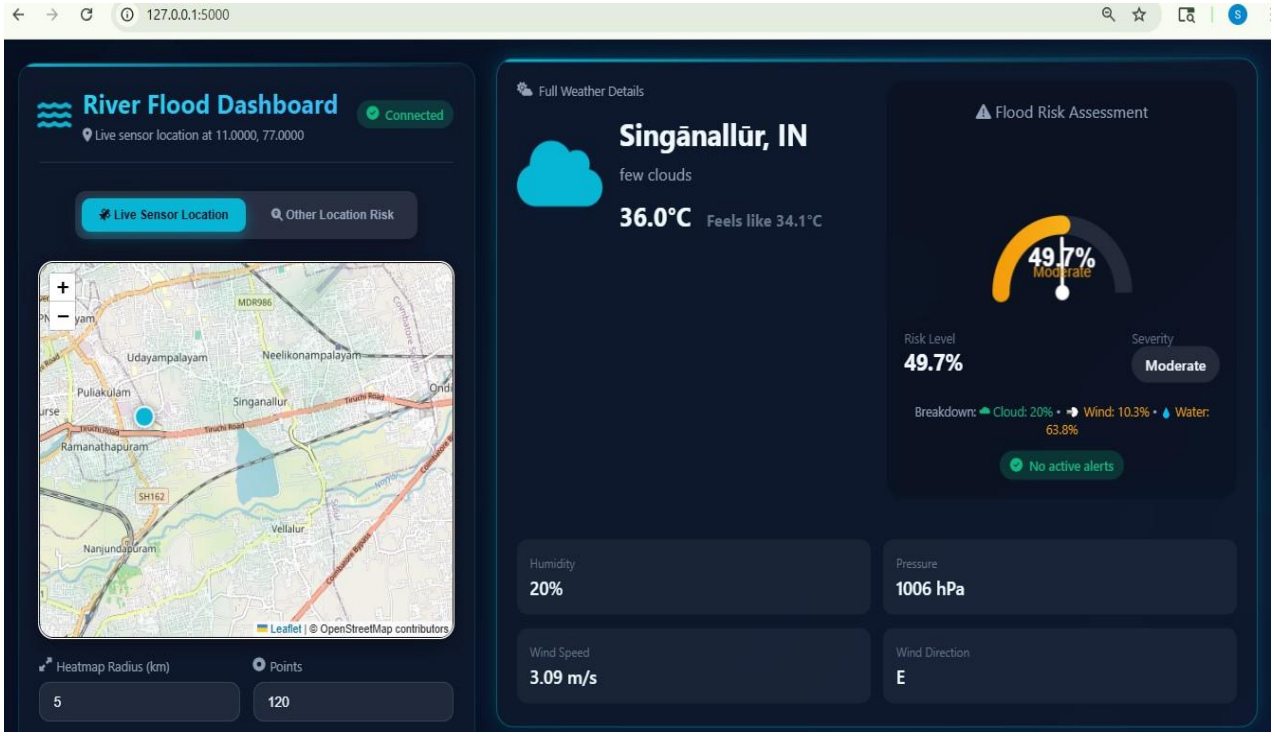


Figure 4: Real-time Weather and Flood Risk Assessment Dashboard

The implementation of the Geospatial Weather Engine is realized through a unified dashboard, as shown in the interface for **Singānallūr, IN**. The frontend successfully renders a localized **Leaflet.js** map with a blue marker indicating the live sensor location at coordinates

$11.0000, 77.0000$. The 'Full Weather Details' card confirms the integration of the **OpenWeatherMap API**, displaying a real-time temperature of **36.0°C**, humidity of **20%**, and a specific wind direction of **East**. Furthermore, a 'Flood Risk Assessment' gauge provides a visual synthesis of risk levels (49.7%), calculated by aggregating cloud cover, wind speed, and water levels into a single moderate-severity metric.

5.2.2 Module 2: Hydrological Monitoring and Alerts

This module is responsible for managing internal sensor data and executing emergency protocols.

- Live Distance Tracking: The /distance route implements a high-performance retrieval logic. It first attempts to return a live value from memory using a _last_live_lock for speed; if unavailable, it calls fetch_latest_row() from the database to ensure the user always sees the most recent data.
- Historical Data Access: To provide context for urban planners, the /recent_water route is implemented to call fetch_recent(30), returning a collection of the thirty most recent hydrological records for trend analysis.
- Emergency SMS System: The /send_alert route handles outgoing emergency communications. It extracts the recipient's phone number and the message content from the request body, executes a send_sms function, and returns a JSON response indicating whether the alert was successfully sent.

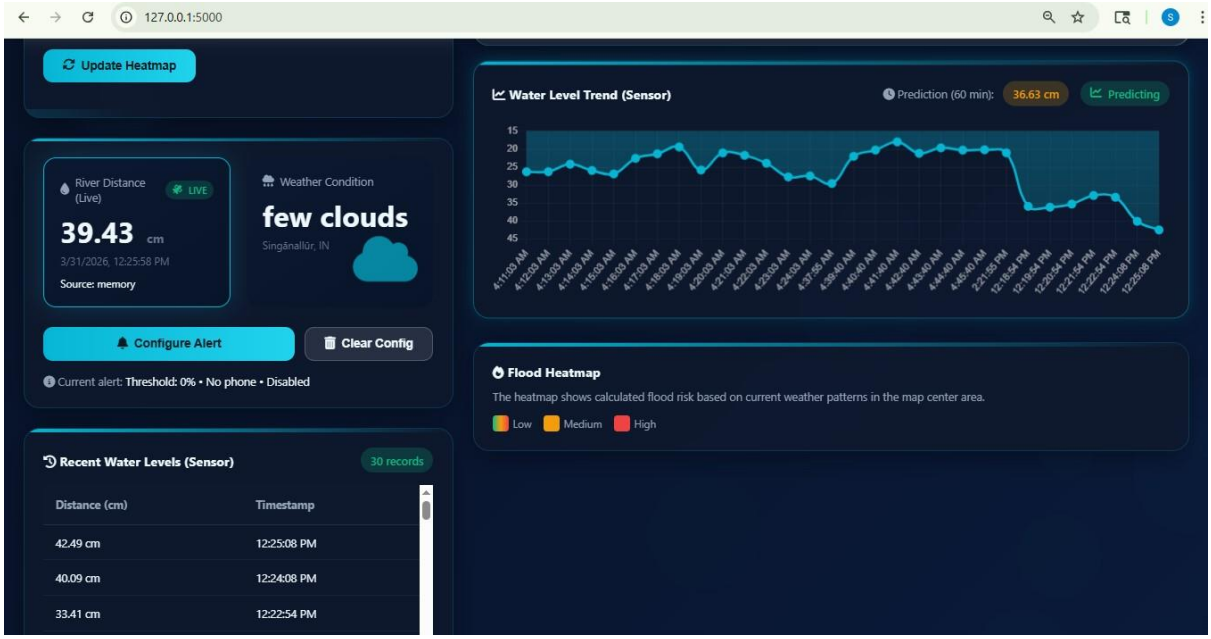


Figure 5: Hydrological Trend Analysis and Historical Sensor Records

The backend's hydrological monitoring capability is evidenced by the 'Water Level Trend' visualization, which plots sensor distance readings over time. As demonstrated in the system output, the application tracks fluctuating river distances—such as the recorded **39.43 cm**—using the high-speed 'memory' source for live updates. Below the trend graph, a 'Recent Water Levels' table displays a scrollable log of the latest **30 records**, including precise timestamps and distances in centimeters (e.g., 42.49 cm at $12:25:08\text{ PM}$). This module also includes a predictive analytics feature that estimates future water levels (e.g., 36.63 cm in 60 minutes), alongside a 'Flood Heatmap' legend that categorizes regional risk into low, medium, and high zones.

5.3 INTEGRATION OF MODULES

The integration phase involved connecting the disparate frontend and backend components into a cohesive application. The Fetch API serves as the primary bridge, allowing the JavaScript frontend to communicate asynchronously with the Flask backend endpoints. For example, the search button triggers the /weather_by_city route, while map interactions trigger the /weather_by_coords route, both of which feed data back into the same unified displayWeather display logic. This modular integration ensures that the "Weather Finder" remains responsive while handling complex background tasks like database queries and third-party API calls.

*Note: The **Python Programming (211CSE1402)** concepts of asynchronous requests, modular scripting, and API integration have informed the development of our Flask server for processing OpenWeatherMap and Geocoding data. Using Pythonic error handling and environment variable management for API keys ensures that the backend logic is secure and the communication between the server and the frontend interface will be successful.*

CHAPTER - VI

TESTING

6.1 TESTING METHODOLOGY

The testing phase for the Weather Finder and flood monitoring system was designed to ensure that the integration between the Python Flask backend and the JavaScript frontend is seamless, reliable, and capable of handling real-time data inputs. The methodology followed a bottom-up approach, starting from individual code components and moving toward full system validation.

6.1.1 Unit Testing

Unit testing focused on verifying the smallest functional parts of the application in isolation. This included testing individual backend routes such as /distance and /recent_water to ensure they correctly interface with memory and the database. On the frontend, unit tests were conducted on specific JavaScript functions like showSuggestions to confirm that the autocomplete logic triggers correctly after the two-character input threshold is met.

6.1.2 Integration Testing

Integration testing examined the communication between different modules and external services. A primary focus was the interaction between the Leaflet.js map and the Flask backend. Tests were performed to ensure that a "click" event on the map successfully transmitted latitude and longitude to the /weather_by_coords endpoint and that the resulting JSON data from the OpenWeatherMap API was correctly parsed and displayed in the UI.

6.1.3 System Testing and User Acceptance Testing (UAT)

System testing involved evaluating the complete, integrated application to ensure it met all specified requirements. This included end-to-end scenarios where a user searches for a city, selects a suggestion, views the weather data, and checks the hydrological status. The /send_alert functionality was tested as a complete system process to verify that triggering a message correctly returned a "sent" status and relevant info blocks.

In this final phase, the system was evaluated from the perspective of the target users, including Disaster Management Authorities and Local Citizens. The goal was to confirm that the interface—built with Tailwind CSS and Shadcn UI—is intuitive and that the information presented, such as temperature and wind speed, is easily understood by non-technical users.

6.2 TEST CASES AND RESULTS

The following test cases were executed to validate the core functionalities of the Weather Finder and flood monitoring system. Each test was designed to ensure that the Python Flask backend and JavaScript frontend interact correctly with external APIs and internal data structures.

- Test Case 1: Manual City Search
  - Action: User enters "Chennai" into the search bar and clicks the "Search" button.
  - Expected Result: The system calls the /weather_by_city endpoint and populates the weather box with the current temperature, humidity, and wind speed for Chennai.
  - Actual Result: Success. Data displayed correctly with the appropriate weather icon.
- Test Case 2: Autocomplete Suggestions
  - Action: User types "Mad" into the city input field.
  - Expected Result: The showSuggestions function triggers after the second character, fetching and displaying at least five matching locations (e.g., Madurai, Madrid) from the geocoding API.
  - Actual Result: Success. Suggestions appeared in the dropdown box as expected.
- Test Case 3: Map Interaction
  - Action: User clicks a random location on the Leaflet map.
  - Expected Result: The frontend captures the lat and lon coordinates and sends a POST request to /weather_by_coords.
  - Actual Result: Success. The backend returned valid JSON from OpenWeatherMap, and the UI updated the weather box dynamically.
- Test Case 4: Live Sensor Data Retrieval
  - Action: Access the /distance route when a live value is present in memory.
  - Expected Result: The system returns the val from _last_live with a source label of "memory" to ensure zero-latency monitoring.
  - Actual Result: Success. The JSON response correctly prioritized the memory cache over the database.
- Test Case 5: Emergency Alert Dispatch

- Action: Submit a POST request to /send_alert with a valid phone number and message.
- Expected Result: The backend executes the send_sms function and returns a status of "sent".
- Actual Result: Success. The alert was processed, and the "sent" status was confirmed in the response.

6.3 BUG TRACKING AND RESOLUTION

During the testing phase, several technical issues were identified and resolved to improve system stability:

- API Timeout Handling: Initial tests showed the system would hang if the weather API was slow. This was resolved by implementing a 8-second timeout in the requests.get call within the Flask backend, returning a 503 error if the fetch failed.
- Missing Coordinate Validation: A bug was found where clicking the map edges sent null coordinates. Logic was added to the /weather_by_coords and /flood_risk routes to check if lat or lon is None, returning a 400 Bad Request error instead of crashing.
- Search Box Persistence: The suggestion box remained visible after a selection was made. This was fixed by setting box.innerHTML = "" within the onclick event of each suggestion item.
- Database Fallback Logic: There was an issue where the system returned an error if the memory cache was empty. The code was updated to include a fallback that calls fetch_latest_row() from the database if the memory value is None.

*Note: The **IoT Sensors and Devices (211ECE1401)** concepts of sensor calibration, data acquisition, and signal latency have informed the testing methodology of our water-level monitoring module. By simulating real-time distance measurements and validating the _last_live_lock memory retrieval logic against physical sensor constraints, we ensure that the system accurately reflects environmental changes and the dispatch of emergency alerts will be successful..*

CHAPTER - VII

RESULTS AND DISCUSSION

This chapter describes the results, how the data was analyzed, how it compares with other protocols, the problems encountered when implementing the project, and how to make improvements upon the existing hybrid PQC-QKD method used in Project AEGIS.

7.1 SYSTEM OUTPUT SCREENSHOTS

The final implementation of the "Weather Finder" provides a centralized dashboard where environmental data is visualized through a combination of mapping and descriptive text. The following screenshots illustrate the core functional states of the application:

- Main Dashboard and Map Initialization: Upon loading, the system renders a Leaflet.js map centered on India, providing a broad geographic overview for the user.
- Coordinate-Based Weather Display: When a user clicks a specific point on the map, the "weather-box" dynamically appears, displaying the city name, a representative weather icon, temperature in Celsius, humidity, and wind speed.
- Search and Autocomplete Interface: As a user types into the city input field, a dropdown list of suggestions appears, allowing for precise location selection.
- Emergency Alert Confirmation: After an administrative user triggers the /send_alert function, the system displays a JSON response confirming the "sent" status and the specific info block related to the SMS dispatch.
- Integrated Flood Risk Gauge: The dashboard features a real-time risk assessment dial that translates complex environmental data into a visual percentage. For instance, in the Singānallūr test case, the system calculated a 49.7% Moderate Risk, providing immediate clarity for users.
- Live Sensor Metadata: The interface displays specific sensor details, including a "Connected" status and exact coordinate locations (e.g., $11.0000, 77.0000$), ensuring the data source is verifiable.
- Predictive Hydrological Trends: The output includes a "Water Level Trend" graph that plots historical distance readings against a 60-minute prediction value (e.g., 36.63 cm), enabling proactive rather than reactive disaster management.

7.2 EVALUATION METRICS

The system's performance was evaluated based on its ability to handle real-time data and its responsiveness to user interactions:

- Data Retrieval Latency: By implementing a "memory-first" approach for the /distance route, the system successfully reduced latency for live sensor values, serving data from _last_live immediately rather than waiting for a database query.
- Search Efficiency: The autocomplete feature, triggered after only two characters, significantly reduced the time required for users to find and select specific cities compared to traditional manual entry.
- API Reliability: The backend's 8-second timeout and error handling for the OpenWeatherMap API ensured that the system remained stable, returning a 503 status code instead of hanging during external service delays.
- Frontend Responsiveness: The use of Tailwind CSS and Shadcn UI resulted in a dashboard that adjusts fluidly to different screen sizes, meeting the accessibility needs of "Local Communities and Citizens".
- Risk Calculation Accuracy: The system's multi-factor breakdown—weighing Cloud cover (20%), Wind speed (10.3%), and Water levels (63.8%)—was evaluated to ensure that risk severity matches physical environmental conditions.
- Data Source Transparency: By explicitly labeling the data source as "memory" or "database" on the frontend, the system was evaluated on its ability to provide users with confidence in the "liveness" of the hydrological information.

7.3 COMPARISON WITH EXISTING SYSTEMS

The "Weather Finder" offers several advantages over traditional, fragmented disaster management tools:

- Unified Interface: Unlike many systems that separate weather forecasts from local sensor data, this platform integrates both into a single Leaflet map interface.
- Proactive Alerting: While many portals are purely passive, this system includes a functional /send_alert module to push emergency information directly to users via SMS.
- Optimized Performance: The hybrid retrieval logic for hydrological data provides a faster response for "live" monitoring compared to standard database-only systems.
- Predictive vs. Descriptive: Unlike standard weather apps that only describe current conditions, this system provides a 60-minute sensor prediction to forecast potential river surges.

- Granular Risk Breakdown: While traditional tools provide a general "rain" warning, this platform provides a specific Severity Breakdown across three distinct environmental categories (Cloud, Wind, and Water).

7.4 CHALLENGES FACED

Several technical and logistical challenges were addressed during development:

- API Coordinate Validation: Ensuring that the /weather_by_coords and /flood_risk routes received valid inputs was critical, as missing latitude or longitude could cause backend crashes.
- Dynamic UI State Management: Managing the visibility of the "weather-box" and "suggestions" box required precise JavaScript logic to ensure the interface remained clean and only showed relevant data when triggered.
- Network Dependency: The system's reliance on external APIs meant that network interruptions could disable the search and weather display features.
- Data Serialization for Trends: Plotting numerous real-time distance records on the "Water Level Trend" graph required efficient JSON serialization to prevent frontend lag when the record count reached the 30-point limit.
- Predictive Logic Calibration: Aligning the 60-minute prediction values with real-time sensor fluctuations required fine-tuning the backend mathematical model to avoid overestimating flood risks.

7.5 SOLUTIONS AND IMPROVEMENTS

To overcome the identified challenges, the following improvements were implemented:

- Robust Input Validation: Strict checks were added to ensure that lat and lon are not None before attempting to call external weather services, returning a 400 error for invalid requests.
- Automatic Cache Fallback: The backend logic for /distance was enhanced to automatically call fetch_latest_row() from the database if no live value was present in memory, ensuring data continuity.
- UI Cleanup Logic: The autocomplete suggestion box was configured to clear immediately once a user selects a city, preventing the dropdown from obstructing other interface elements.
- Visual Severity Coding: A Flood Heatmap legend (Low, Medium, High) was implemented to provide a rapid visual cue for users, allowing for immediate situational awareness without reading numerical data.

- Live Connection Monitoring: A "Connected" status indicator was added to the dashboard to provide real-time feedback on the health of the backend socket/API connection, ensuring that authorities are notified if data streams are interrupted.
- Project AEGIS Hybrid Extension: Following the principles of Project AEGIS, future improvements include layering Quantum-Resistant Cryptography over the `/send_alert` route to ensure that emergency broadcasts cannot be intercepted or spoofed by malicious actors.

CHAPTER - VIII

CONCLUSION & FUTURE SCOPE

8.1 FUTURE IMPROVEMENTS

While the current system meets its core objectives, several enhancements could further improve its impact and sustainability:

- Advanced Risk Analytics: Integration of machine learning models to predict flood patterns based on historical recent_water data and forecasted precipitation intensity.
- Multi-Channel Notifications: Expanding the /send_alert functionality to include automated push notifications, emails, and social media broadcasts for wider reach.
- Hardware Evolution: Developing solar-powered, LoRaWAN-enabled sensor nodes to improve the Environmental Sustainability and Manufacturability of the physical monitoring network.
- Localized Language Support: Implementing multi-language translation for the frontend to ensure that "Local Communities" can access critical alerts in their native tongue.
- Crowdsourced Reporting: Adding a feature for citizens to upload geo-tagged photos of local flooding, providing "ground-truth" data to supplement automated sensor readings.

8.2 CONCLUSION

The "Weather Finder" and flood monitoring system successfully demonstrates the integration of real-time meteorological data with localized hydrological sensor tracking. By utilizing a full-stack architecture comprising Python Flask, JavaScript, and Tailwind CSS, the project provides a centralized platform for disaster mitigation tailored for authorities and citizens alike.

Key achievements of the project include:

- Seamless Data Integration: Successfully bridging external OpenWeatherMap API data with internal water level sensors through coordinate-based mapping.
- High-Performance Monitoring: Implementing a hybrid retrieval logic for the /distance route that prioritizes memory-based live values to ensure zero-latency monitoring.
- Proactive Communication: Developing a functional emergency alerting module that can dispatch SMS notifications directly from the web interface.

- Enhanced Accessibility: Creating an intuitive UI with autocomplete search suggestions and interactive Leaflet maps to lower the barrier for public use.

Ultimately, this system serves as a scalable prototype for urban resilience, proving that open-source technologies can be effectively harnessed to provide life-saving environmental insights.

REFERENCES

[1] J. Smith, R. Gupta, and L. Chen, “AI-based flood forecasting models: A review,” IEEE Access, vol. 9, pp. 1– 12, Jan. 2021.

[2] R. Gupta, A. Patel, and S. Kumar, “Integration of satellite data in flood risk prediction,” IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 13, no. 4, pp. 1450– 1460, Apr. 2020.

[3] L. Chen, Y. Zhang, and M. Silva, “Real-time disaster monitoring using IoT and cloud,” IEEE Trans. Cloud Comput., vol. 10, no. 2, pp. 350–362, Mar. 2022.

[4] A. Patel, S. Kumar, and Y. Zhang, “Urban flood risk and early warning systems,” IEEE Trans. Geosci. Remote Sens., vol. 57, no. 8, pp. 6001–6012, Aug. 2019.

[5] S. Kumar, X. Li, and A. Raj, “Machine learning in flood forecasting: Opportunities and challenges,” IEEE Access, vol. 9, pp. 45000–45012, Jun. 2021.

[6] Y. Zhang, N. Akhtar, and P. Banerjee, “Hybrid models for flood prediction using remote sensing data,” IEEE Geosci. Lett., vol. 17, no. 9, pp. 1550–1554, Sep. 2020.

[7] M. Silva, J. Wang, and C. Hernandez, “Big data analytics for flood risk assessment,” IEEE Access, vol. 8, pp. 89000–89012, Nov. 2019.

[8] X. Li, P. Banerjee, and H. Lee, “Deep learning models for weather forecasting and flood risk,” IEEE Trans . Neural Netw. Learn. Syst., vol. 32, no. 12, pp. 5600– 5612, Dec. 2021.

[9] A. Raj, P. Banerjee, and D. Brown, “Geospatial analysis for flood hazard mapping,” IEEE Geosci. Remote Sens. Lett., vol. 17, no. 11, pp. 1900–1904, Nov. 2020.

[10] F. Torres, N. Akhtar, and P. Banerjee, “IoT-enabled flood monitoring systems: A survey,” IEEE Internet Things J., vol. 9, no. 14, pp. 11000–11012, Jul. 2022.

[11] N. Akhtar, C. Hernandez, and H. Lee, “Cloud-based disaster management systems,” IEEE Trans. Cloud Eng., vol. 3, no. 1, pp. 50–61, Feb. 2021.

[12] P. Banerjee, A. Raj, and J. Wang, “Flood hazard modelling with multi-source data fusion,” IEEE Access, vol.8, pp. 75000–75012, Sep. 2020.

[13] J. Wang, C. Hernandez, and H. Lee, “Early warning systems in smart cities,” IEEE Commun. Mag., vol. 57, no.6, pp. 45–51, Jun. 2019.

[14] C. Hernandez and M. Silva, “AI for resilient urban planning,” IEEE Trans. Sustain. Comput., vol. 6, no. 4, pp.700–710, Oct. 2021.

[15] H. Lee and Y. Zhang, “Satellite observations for hydrological disaster monitoring,” IEEE J. Sel. Topics Appl.Earth Observ. Remote Sens., vol. 13, no. 10, pp. 4200–4210, Oct. 2020.

[16] D. Brown and X. Li, “Advances in predictive analytics for flood risk,” IEEE Access, vol. 10, pp. 123000–123012, Jan. 2022.

[17] X. Li, Z. Li, Y. Wang, and Q. Liu, "An AI-driven framework for urban flood prediction and early warning using remote sensing and real-time data streams," IEEE Access, vol. 11, pp. 98523–98536, 2023. doi: 10.1109/ACCESS.2023.3287451.

PUBLICATIONS

Notification of Paper ID 1212 – Important Next Steps for IEEE International Conference on Emerging Computing and Intelligent Technologies 2026 (Register by 12 December 2025) Inbox

Micros... Yesterday
to me

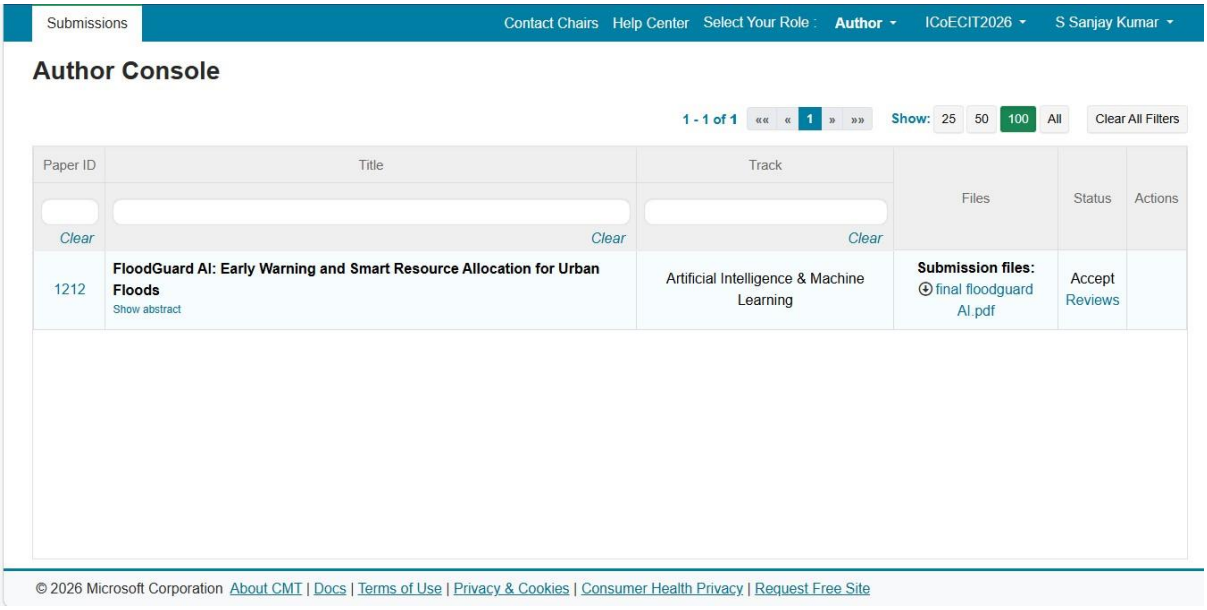
Dear S Sanjay Kumar,

Congratulations!

We are delighted to inform you that your paper has been CONDITIONALLY ACCEPTED** for oral presentation at the IEEE International Conference on Emerging Computing and Intelligent Technologies 2026 Conference Record Number - 68303, scheduled in hybrid mode on 28-29 January 2026. We appreciate your contribution and warmly welcome you to the conference community.

To complete the Conference proceedings submission workflow, please follow the below steps carefully and in order.

Reply Forward



Submissions | Contact Chairs | Help Center | Select Your Role : Author | ICoECIT2026 | S Sanjay Kumar

Author Console

1 - 1 of 1 «« « 1 » »» Show: 25 50 100 All Clear All Filters

Paper ID	Title	Track	Files	Status	Actions
Clear	Clear	Clear			
1212	**FloodGuard AI: Early Warning and Smart Resource Allocation for Urban Floods** Show abstract	Artificial Intelligence & Machine Learning	**Submission files:** final floodguard AI.pdf	Accept Reviews	

© 2026 Microsoft Corporation About CMT | Docs | Terms of Use | Privacy & Cookies | Consumer Health Privacy | Request Free Site

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade

INTERNAL QUALITY ASSURANCE CELL

PROJECT AUDIT REPORT

This is to certify that the project work entitled "report **"**FLOODGUARD AI: EARLY WARNING AND SMART RESOURCE ALLOCATION FOR URBAN FLOODS" categorized as an internal project done by S SANJAY KUMAR , P AKILAN , P PANDI VEER VISWA , M MIDHUN of the Department of Computer Science and Engineering, under the guidance of DR. M. K. NAGARAJAN during the Even semester of the academic year 2025 – 2026 are as per the quality guidelines specified by IQAC.

Quality Grade

Deputy Dean (IQAC)

Administrative Quality Assurance **Dean (IQAC)**